



4300 Final Project: Risc-V Benchmarking

Carson Quesada
Derek Jacoby
Demajay Francis
Cole Pedersen



Project Overview

We want to benchmark a set of Risc-V cores, measuring speed and efficiency and then compare them.

What is Risc-V?

- *RISC-V is an open-source, modular instruction set architecture (ISA) based on the reduced instruction set computer design.*

Why benchmark cores?

- *To measure how different RISC-V core designs impact performance and power efficiency.*



Project Overview

Goal:

Benchmark different **RV32IMC** soft cores implemented on an FPGA to compare their **power** and **performance** characteristics.

Cores Tested:

- PicoRV32 (1 stage)
- Ibex (2 stage)
- VexRiscV (5 stage)
- CORE-V CV32E40P (4 stage)

Why:

To understand how pipeline depth, instruction set features, and core design affect **energy efficiency**, **CPI**, and **execution throughput**.



Deliverables

Metrics:

- Cycle count
- Instruction count
- CPI
- Throughput Latency
- Power usage

Operations we will take measurements on

- Addition/subtraction
- Multiplication/Division
- Load/Store
- Branch instructions
- Bit Shift operations

Hardware

FPGA Board: Nexys A7 (Artix-7 FPGA)

Clock Frequency: 100 MHz

Peripherals:

- UART TX (115200 baud) for optional debug output
- GPIO for simple output control

Power Measurement:

- External power meter connected to FPGA supply rail
- Each test ran for 5–10s for stable readings

Memory Map:

Address Range	Function
0x8000_0000 – 0x8000_FFFF	64 KB BRAM (program + data)
0x1000_0000	UART TX (write)
0x1000_0004	GPIO output (LED/button)



Software

Toolchain:

xPack RISC-V GCC 11.5.0
(riscv-none-elf-gcc)

Vivado: Used for synthesis,
implementation, and bitstream generation.

Putty: Used for viewing output

Tool Flow:

C source (.c)



compile & link (GCC +
link.ld)



ELF file (.elf)



convert to Verilog HEX
(objcopy)



\$readmemh() loads program
into BRAM



Test Case Design

- Each test runs **bare-metal** (no OS) and loops indefinitely after startup.
- Programs were linked to start at **0x80000000**, matching the cores' reset address.
- Each test compiled to its own .hex file and loaded individually.

Test Programs Implemented:

1. ALU Throughput
2. Multiply/Divide Latency
3. Load/Store Memory Access
4. Branch Instructions (Always, Never, 50/50)
5. Bit-Shift Operations



Tool Flow

```
riscv-none-elf-gcc -march=rv32imc -mabi=ilp32 \  
-O0 -ffreestanding -nostdlib -T link.ld -Wl,-e,main \  
-o main.elf main.c -lgcc
```

Creates main.elf → an executable file containing your compiled machine code and memory layout.

```
riscv-none-elf-objcopy -O verilog --verilog-data-width=4 \  
--change-addresses=0x48000000 main.elf main.hex
```

The second command converts that ELF into a Verilog-compatible main.hex file that the FPGA loads into BRAM.

Code

```
// Short precise speed test (CPI measurement)
static void addition_speed_test(void) {
    volatile uint32_t a = 1, b = 2, c = 3;
    uint64_t c0 = rdcycle64();
    uint64_t i0 = rdinstret64();

    for (int k = 0; k < 100000; k++) {
        a += b + c;
        b += a;
        c += b;
    }

    uint64_t c1 = rdcycle64();
    uint64_t i1 = rdinstret64();

    uint64_t cycles = c1 - c0;
    uint64_t insts = i1 - i0;
    uint64_t cpi_x10000 = (cycles * 10000ULL) / insts; // scaled integer CPI

    uart_puts("Addition speed test finished.\r\n");
    uart_puts("Cycles: "); uart_putu(cycles); uart_puts("\r\n");
    uart_puts("Insts: "); uart_putu(insts); uart_puts("\r\n");
    uart_puts("CPI x10^4: "); uart_putu(cpi_x10000); uart_puts("\r\n\r\n");
}

// Long loop for power measurement (no UART while running)
static void addition_power_test(void) {
    volatile uint32_t a = 1, b = 2, c = 3, d = 4, e = 5, f = 6, g = 7, h = 8;
    const uint32_t ITER = 500000000; // ~5 s @100 MHz

    for (uint32_t k = 0; k < ITER; k++) {
        a = b + c; d = e + f; g = h + a; b = c + d;
        c = d + e; e = f + g; f = g + h; h = a + b;
        __asm__ volatile("" : "+r"(a), "+r"(b), "+r"(c),
            "+r"(d), "+r"(e), "+r"(f), "+r"(g), "+r"(h));
    }
}
```

Code

```
// --- ALU TEST ---
static void alu_test(void) {
    uart_puts("Running ALU test...\r\n");
    volatile uint32_t a = 1, b = 2, c = 3, d = 4;
    const uint32_t ITER = 100000000;
    for (uint32_t k = 0; k < ITER; k++) {
        a += b;
        c += d;
        b ^= c;
        d ^= a;
        __asm__ volatile("" ::: "memory");
    }
    uart_puts("ALU test complete.\r\n");
}

// --- MUL/DIV TEST ---
static void muldiv_test(void) {
    uart_puts("Running MUL/DIV test...\r\n");
    volatile uint32_t a = 3, b = 7, c = 11;
    const uint32_t ITER = 100000000;
    for (uint32_t i = 0; i < ITER; i++) {
        a = a * b;
        b = a / 3u;
        c = b % 5u;
        a ^= c + 1u;
        __asm__ volatile("" ::: "memory");
    }
    uart_puts("MUL/DIV test complete.\r\n");
}
```

```
// --- LOAD/STORE TEST ---
#define N 1024
static uint32_t buf[N];
static void loadstore_test(void) {
    uart_puts("Running Load/Store test...\r\n");
    volatile uint32_t sum = 0;
    const uint32_t ITER = 1000;
    for (uint32_t t = 0; t < ITER; t++) {
        for (int i = 0; i < N; i++) buf[i] = i * 3;
        for (int i = 0; i < N; i++) sum += buf[i];
        __asm__ volatile("" ::: "memory");
    }
    uart_puts("Load/Store test complete.\r\n");
}

// --- BRANCH TEST ---
static void branch_test(void) {
    uart_puts("Running Branch test...\r\n");
    volatile uint32_t x = 0, t = 0;
    const uint32_t ITER = 100000000;
    for (uint32_t i = 0; i < ITER; i++) {
        if (1) x++;           // always taken
        if (0) x--;           // never taken
        t ^= 1;
        if (t) x += 3; else x -= 2; // 50/50
        __asm__ volatile("" ::: "memory");
    }
    uart_puts("Branch test complete.\r\n");
}

// --- SHIFT TEST ---
static void shift_test(void) {
    uart_puts("Running Shift test...\r\n");
    volatile uint32_t x = 0x13579BDFu, y = 1, z = 0;
    const uint32_t ITER = 100000000;
    for (uint32_t i = 0; i < ITER; i++) {
        z = x << (y & 31);
        z ^= x >> ((y + 7) & 31);
        z += (int32_t)x >> ((y + 13) & 31);
        y += z | 1u;
        __asm__ volatile("" ::: "memory");
    }
    uart_puts("Shift test complete.\r\n");
}
```



Code

```
int main(void) {

    uart_puts("Press CONTINUE for addition speed test or SKIP for power test.\r\n");

    while (1) {
        if (CONTINUE_BTN & 1) {
            wait_for_press_release(&CONTINUE_BTN);
            addition_speed_test();
            break;
        } else if (SKIP_BTN & 1) {
            wait_for_press_release(&SKIP_BTN);
            break;
        }
    }

    uart_puts("Press CONTINUE for addition power test or SKIP to finish.\r\n");

    while (1) {
        if (CONTINUE_BTN & 1) {
            wait_for_press_release(&CONTINUE_BTN);
            addition_power_test();
            uart_puts("Addition power test complete. Measure power now.\r\n");
            break;
        } else if (SKIP_BTN & 1) {
            wait_for_press_release(&SKIP_BTN);
            break;
        }
    }

    uart_puts("Program done. Press reset to rerun.\r\n");
    while (1) __asm__ volatile("wfi");
}
```

Putty Output - Cycle Count & Instruction

```
Booting...
Press CONTINUE for addition speed test or SKIP for power test.
Addition speed test finished.
cycles: 0000000000A037E5
insts: 0000000000186A0A
Press CONTINUE for addition power test or SKIP to finish.
Program done. Press reset to rerun.
Booting...
Press CONTINUE for addition speed test or SKIP for power test.
Press CONTINUE for addition power test or SKIP to finish.
Addition power test complete. Measure power during the run.
Program done. Press reset to rerun.
```





Results

Core	Test Type	Cycles ($\times 10^6$)	Insts ($\times 10^6$)	CPI	Power (mW)
PicoRV32 (1 stage)	Addition Speed	X	X	X	X
Ibex (2 stage)	Addition Speed	x	x	x	x
VexRiscv (5 stage)	Addition Speed	x	x	x	x
CV32E40P (4 stage)	Addition Speed	x	x	x	x



Challenges

- Address mismatch in .hex output (@20000000 → @80000000).
- Vivado hides non-HDL files unless added manually.
- We might not be able to use the same .hex file for testing with the different cores